

Capítulo 4: Funciones y Control de Flujo como Expresiones

1. Explicación Teórica: La Supremacía de la Expresión

Este capítulo marca la inmersión en la Programación Funcional (PF), que, junto con la Programación Orientada a Objetos (POO) pura, forma el paradigma híbrido de Scala. Para un desarrollador de Scala, las funciones y el control de flujo se conceptualizan bajo un principio fundamental: la **Programación Orientada a Expresiones** (*Expression-Oriented Programming*).

Funciones como Ciudadanos de Primera Clase

En la PF, las funciones no son meras rutinas o subprogramas; son **valores de primera clase**. Esto significa que una función puede ser:

1. Asignada a una variable.
2. Pasada como argumento a otras funciones (Funciones de Orden Superior o HOFs).
3. Devuelta como resultado de otra función.

Todo Devuelve un Valor

En Scala, **casi todas las estructuras se evalúan y devuelven un valor** (son expresiones), a diferencia de las *instrucciones* (*statements*) que simplemente ejecutan una acción sin retornar un resultado.

La consecuencia más significativa de este enfoque es el **Retorno Implícito**: el valor de una función o bloque de código es automáticamente el valor de la **última expresión evaluada** dentro de dicho bloque. Esto elimina la necesidad de utilizar la palabra clave `return` en la mayoría de los casos.

2. Sintaxis y Uso de Funciones

Definición Básica y Retorno Implícito

La definición de funciones utiliza la palabra clave `def`. La sintaxis especifica el nombre, los parámetros de entrada con su tipo obligatorio, y opcionalmente el tipo de retorno, seguido del signo de igualdad '=' que indica que se devuelve un valor.

Característica	Sintaxis Gramatical
Definición de Función	<code>def nombre(parámetro: Tipo, ...): TipoRetorno = expresión</code>
Retorno Implícito	La última línea (expresión) dentro del cuerpo es el valor de retorno.

Ejemplo de Función (Cálculo Financiero)

Este ejemplo ilustra el retorno implícito y la capacidad de las funciones de encapsular lógica compleja.

```

1  /**
2   * Calcula el interés compuesto anual de un préstamo.
3   * No requiere 'return'; la última línea es el valor de retorno implícito.
4   * @param principal Monto inicial.
5   * @param tasaInteres Tasa anual (ej: 0.05 para 5%).
6   * @param anios Número de años.
7   */
8  def calcularInteres(principal: Double, tasaInteres: Double, anios: Int):
    Double = {
9      // El compilador infiere que este bloque retornará un Double
10
11      // Expresión de cálculo (la última línea del bloque)
12      principal * math.pow(1 + tasaInteres, anios)
13  }
14
15  // Uso:
16  val inversionInicial = 1000.0
17  val valorFinal = calcularInteres(inversionInicial, 0.05, 10)
18  // valorFinal: 1628.8946267774424 (Retorno implícito del cálculo)

```

Funciones Anónimas y Shorthand

Las **funciones anónimas** (o *lambdas*) son funciones sin nombre, utilizadas comúnmente como argumentos de otras funciones.

```

1  // Forma explícita (similar a Function1 trait)
2  val incrementador: Int => Int = (x: Int) => x + 1
3
4  // Forma idiomática (el compilador infiere el tipo)
5  val duplicador = (x: Int) => x * 2
6
7  // Forma más corta usando placeholder (_) si el argumento se usa una vez
8  val triplicador = _ * 3

```

```

9
10 // Uso:
11 val resultado = triplicador(7) // 21

```

3. Control de Flujo como Expresiones

Las estructuras de control tradicionales (`if-else` , bloques de código) se utilizan para **producir un valor** en el contexto funcional de Scala.

If/Else como Expresión

El `if-else` es una expresión que devuelve un valor que puede ser asignado directamente a un `val` inmutable.

Comparativa: Java vs. Scala	Java (Instrucción / Ternario)	Scala (Expresión)
Declaración	String status = (edad >= 18) ? "Adulto" : "Menor"; (Usa ternario)	val status = if (edad >= 18) "Adulto" else "Menor"

Ejemplo de Código (Lógica de Negocio)

```

1 // Definición inmutable de la tasa de comisión
2 def obtenerTasaComision(volumenVentas: Double): Double =
3   val tasa =
4     if (volumenVentas > 50000.0) then
5       0.05 // 5% para ventas altas
6     else if (volumenVentas > 10000.0) then
7       0.03 // 3% para ventas medias
8     else
9       0.01 // 1% para ventas bajas
10
11 // La última línea (tasa) se convierte en el valor de retorno implícito.
12 tasa
13
14 val comisionVendedor =
15   obtenerTasaComision(60000.0) * 60000.0
16 // comisionVendedor: 3000.0 (Calculado mediante una expresión if-else)

```

Bloques de Código como Expresiones

Un bloque de código delimitado por llaves `{}` se comporta como una expresión, retornando el valor de su última línea. Esto es útil para encapsular variables locales (`val`) temporales que se usan solo para calcular el valor final.

```
1  val resultadoComplejo: Double = {
2    // Inicialización de valores locales (solo accesibles dentro de este
    bloque)
3    val valorBase = 1000.0
4    val ajusteImpuesto = 1.18
5
6    // La última expresión es el valor devuelto por el bloque
7    valorBase * ajusteImpuesto
8  } // resultadoComplejo: 1180.0
```

4. Iteración Funcional: HOFs y For Comprehensions

En Scala, se **evita la iteración imperativa** mediante bucles `for` o `while` al estilo Java/Python, ya que a menudo requieren el uso de variables mutables (`var`). La **forma idiomática** de trabajar con colecciones es a través de las **Funciones de Orden Superior (HOFs)**.

Funciones de Orden Superior (HOFs)

HOFs como `map` , `filter` , `fold` y `reduce` permiten transformar colecciones de manera declarativa y segura, devolviendo nuevas colecciones inmutables.

Tarea	Código Imperativo (Java/Python style)	Código Funcional Idiomático (Scala)
Filtrar y Mapear	Bucle explícito con asignación mutable.	Composición de HOFs: <code>list.filter(...).map(...)</code>
Reducción	Inicializar un acumulador (<code>var</code>).	Uso de <code>fold</code> o <code>reduce</code> .

Ejemplo de Código (Procesamiento de Logs)

```
1  // Lista inmutable de códigos de estado HTTP
2  val codigosHTTP = List(200, 404, 500, 201, 403, 200, 301)
3
4  // 1. Uso de filter (predicado) para encontrar errores
```

```

5  val erroresCliente = codigosHTTP.filter(codigo => codigo >= 400 && codigo
   < 500)
6  // List(404, 403)
7
8  // 2. Uso de map para convertir cada código a su descripción (String)
9  val descripciones = erroresCliente.map(codigo => s"Error $codigo")
10 // List("Error 404", "Error 403")
11
12 // 3. Uso de foldLeft (reduce) para sumar la cantidad total de errores
13 // 0 es el valor inicial (seed), (total, _) es la función acumuladora
14 val conteoErrores = erroresCliente.foldLeft(0)((total, _) => total + 1)
15 // 3 (Se puede simplificar a .size, pero muestra el uso de fold/reduce)

```

For Comprehensions (Azúcar Sintáctico)

Una **for comprehension** no es un bucle; es azúcar sintáctico para encadenar llamadas a `map`, `flatMap` y `filter`. Se utiliza para operaciones complejas donde la composición directa con `map/flatMap` se volvería difícil de leer. El uso de `yield` es obligatorio para que la comprensión retorne una colección.

```

1  val listaNumeros = List(1, 2, 3)
2  val listaLetras = List("a", "b")
3
4  // Generar todas las combinaciones (producto cartesiano) de numero y letra
5  val combinaciones =
6    for {
7      numero <- listaNumeros // flatMap implícito
8      letra <- listaLetras    // map implícito
9    } yield (numero, letra) // Resultado esperado (Tuple)
10
11 // combinaciones: List((1,a), (1,b), (2,a), (2,b), (3,a), (3,b))

```

5. Best Practices (*The Scala Way*)

El uso idiomático de Scala exige priorizar las abstracciones funcionales:

- **Inmutabilidad y Expresiones:** Siempre que sea posible, **modela la lógica de negocios utilizando expresiones** que devuelven un valor (`if-else`, `{bloques}`, `match`) y asigna el resultado a un `val` (valor inmutable), evitando así el estado mutable.
- **HOFs sobre Imperativo:** Para cualquier manipulación de colecciones (listas, conjuntos, mapas), utiliza `map`, `filter`, `flatMap` o `fold`. La recursión es el mecanismo preferido en lugar de los bucles `while`.

- **Funciones Anónimas y Concisión:** Aprovecha la inferencia de tipos para mantener las declaraciones de funciones anónimas concisas, utilizando *placeholders* (`_`) cuando sea apropiado, especialmente en llamadas a HOFs.

Comparativa con Java/Python: En Scala, la lógica es declarativa. No estás diciendo *cómo* iterar (instrucción mutable), sino *qué* transformación aplicar a los datos (expresión inmutable). Esto se asemeja a describir una receta matemática donde el resultado está garantizado por la composición de sus partes, sin pasos intermedios que puedan arruinar el plato.